

01

Client and server

The machines your app runs on, and which of them you control

Most of what goes wrong at the boundary of an AI-assisted project traces to one mistaken picture: the app as a single thing in a single place, where a rule written anywhere applies everywhere. Every web app is in fact at least two programs — one running on machines you control, one running on machines you don't — and a rule binds only on your side of that line. This chapter builds the two-sided picture; the chapters on authorization and secrets stand directly on it.

The model

Every piece of code runs on one specific machine

The code in your project is a pile of text files, and text on its own does nothing: to have any effect it has to run, as a process, on one specific computer. So for every piece of your project there’s a question with a definite answer: *which machine does this run on?* The answer decides who controls that piece, and most of this chapter follows from asking it consistently.

The machines involved are few. Your laptop runs the code while you’re building — that’s what “running it locally” means. A server runs it for real users; a server is a computer like any other, usually rented, sitting in someone’s data centre, and chapter 5 is about it. And the visitor’s own browser runs a third portion, which is the one this chapter turns on.

The visitor’s side

When someone opens your app, their browser downloads your interface code — the pages, the buttons, the checks around them — and runs it on their machine. From that moment that copy is theirs to inspect and change, since it’s running on hardware you have no access to.

Every browser ships with developer tools that make this concrete: a panel that shows the downloaded code, a console that runs arbitrary commands against the open page, and a network tab that lists every request the page sends. This is standard equipment for building and debugging the web.

Your side is the server and the database behind it. Visitors can send requests to your side, and that is the whole of what they can do to it — which is why it’s the only place a rule binds.

DIAGRAM

two sides separated by a dashed line labelled “the wire”. Visitor’s side: browser holding “your interface code (their copy)”, devtools, console. Your side: server, then database. A single arrow crosses the line, labelled “requests – the only traffic”.

Anyone can send a request by hand

The interface talks to your server by sending requests: small structured messages saying “create this”, “give me that”. Your form composes the request from what the user typed, after running whatever checks you gave it.

Nothing obliges anyone to use the form. The network tab shows exactly what request a button sends, and one copy-paste into the console re-sends it — with the fields changed, or the parts they don’t want removed. The server can’t distinguish a request your form composed from one typed by hand, because on arrival they’re the same kind of message.

A check that runs in the browser therefore executes only when the visitor chooses to run it. It serves honest users by catching their mistakes before a round trip; the rule itself has to live on

your side, where every request passes through it no matter who composed it.

The vocabulary of a running machine

Four terms cover most of what an assistant will say about the machine your code runs on. They’re worth recognizing rather than mastering.

A **process** is one running program. Your app running locally is a process; stopping it is why the page goes dead.

The **filesystem** is the machine’s local storage. Code can read and write files on the machine it runs on — and only that machine, which is why a file saved locally doesn’t exist in production.

An **environment variable** is a named value a process is handed at startup, kept outside the code. It’s where configuration and keys live, and chapter 7 is about what belongs there.

A **port** is a numbered door on a machine where a process listens for requests. `localhost:3000` means “the process listening on door 3000 of this machine” — the address of your own app while you build it.

What people mean by “a stack”

A stack is a named set of choices: which language and framework on the server, which database, which host. The word matters to you for one practical reason — assistants have defaults, advice transfers within a stack much better than across, and knowing your stack’s names lets you ask questions that get answers about your actual setup rather than about the general case.

PART

What goes wrong

The three entries here are instances of one failure: something the server needed to do itself, done only on the visitor’s side.

01

Client and server

1 Validation on the client only

the form checks the input and the server accepts anything.

ASK “For each input a user can submit, state every place it’s validated: browser form, server, database, or nowhere. Present it as a table, with ‘nowhere’ written out wherever that’s the answer.”

CHECK send the request directly with a value the form would reject, and see whether it lands.

TELL *rules present in the form component and absent from the request handler.*

2 The unvalidated upload

the file picker’s type filter is the only filter.

ASK “For each upload in my project, state where file type and file size are enforced: the browser, the server, the storage service’s own rules, or nowhere.”

CHECK give a file the wrong extension for what it is and upload it; then upload one far larger than the interface implies is welcome.

TELL *an upload where the accepted-types list on the input element is the only mention of type, and size is mentioned nowhere.*

3 The value computed on their side

a number the server should have worked out itself arrives from the browser and is believed.

ASK “List every value the server receives from the browser that it could compute or look up itself — totals, prices, quantities against stock, roles, the current user’s id. For each, say whether the server recomputes it or trusts what arrived.”

CHECK send the request with the value changed — a total of zero, a role of admin — and read the row that results.

TELL *a total, a price, a discount, a role, or the current user’s id sent in the request body and written straight to a row.*

THE DIRECT TEST

Open the network tab, use your own form once, and find the request it sent. Copy it into the console, change one value to something the form forbids, and send it. A rejection tells you which server-side rule caught it; success tells you the rule you thought you had lives only in the form. The whole test takes about ten minutes.

PART

Going deeper

These prompts are for your assistant, and each comes with a note on what a good response looks like. If you get prose instead of the artifact, re-ask insisting on the list.

D1 · Map my app

GROUNDING EXPLANATION

List every distinct program that runs when someone uses my app. For each: what it’s called in my project, which machine it runs on, who controls that machine, and what it can reach directly — files, the database, other services. Include anything my hosting platform runs on my behalf, and anything that runs partly in one place and partly in another.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response is a short table where every row names a machine and who controls it. If it lists files or folders instead of running programs, re-ask — the question is about processes, not code layout.

D2 · The validation audit

AUDIT

For every input a user can submit anywhere in my app — form fields, uploads, anything editable — state every place it’s validated: browser form, server, database, or nowhere. One row per input, as a table, with ‘nowhere’ written out explicitly wherever that’s the truth. Don’t summarize; I want the full list.

WHAT A GOOD RESPONSE LOOKS LIKE

A good response has a row per input and at least a few honest “nowhere”s or “browser only”s — a table that’s all green on the first pass deserves suspicion, and a follow-up: “for the three most sensitive inputs, show me the exact request that would bypass the form.”

D3 · The resend

BUILD A TOY

I want to test one of my own forms from the browser console. Walk me through it: open the network tab, submit the form, find the request, copy it as a fetch call, and re-send it with one value changed to something the form wouldn’t allow. Tell me exactly what to click and paste, and then help me read the response and check what actually reached the database.

WHAT A GOOD RESPONSE LOOKS LIKE

This takes about twenty minutes and makes the chapter concrete on your own project. Either ending is a good outcome: a named server-side rule that caught it, or a finding worth fixing.

WHERE THIS CONNECTS

Chapter 6 · Authorization applies this picture to who-can-see-what — its four-places table is this chapter’s boundary with names on it. **Chapter 7 · Secrets and configuration** applies it to keys, where crossing the line is a one-way trip. **Chapter 3 · Source of truth** picks up what happens to data once both sides hold copies of it. **Chapter 8 · Injection** covers the other thing a request can carry: text that becomes code on your side. **Chapter 12 · Verification** is the method every going-deeper section in this series uses; if you’re already shipping, read it next.

D4 · Where does this run?

PREDICTION QUIZ

Quiz me on my own project. One item at a time, name a piece of it — a file, a function, a check, a key — and ask me which machine it runs on (or is sent to) and who controls that machine. Wait for my answer, then correct me with the reason. Include at least one piece that exists on both sides, and one that I probably think is private but ships to the browser.

WHAT A GOOD RESPONSE LOOKS LIKE

Commit to each answer before reading the correction — a wrong prediction points at the specific part of the picture that needs fixing.